

เริ่มโครงการและวางฐาน

จากระบบ Monolithic → โครงสร้าง OOP ที่ทดสอบได้และปลอดภัยขึ้น

15 SP

Story Points ปิดครบ

5 Issues

SPM-6 ถึง SPM-10

5/5

PyTest ผ่าน

ภาพรวมสปรินต์

ตั้งกฎบัตร เข้าใจระบบเดิม ออกแบบพิมพ์เขียว และ refactor พร้อม automated test

W1-W4

ช่วงงาน

15 SP

ปิดครบ

5

Issues

5/5

Tests PASS

ทีมและความรับผิดชอบ

ภาณุวัฒน์ คำคำ

บริหารภาพรวม พัฒนา และจัดการหลักฐานโครงการ

PM / Developer

เอกพันธ์ ทศทิศรังสรรค์

ออกแบบและยืนยันผลทดสอบ

QA / Tester

ณัฐภาพ สายหล้า

ออกแบบสถาปัตยกรรมและตรวจคุณภาพ

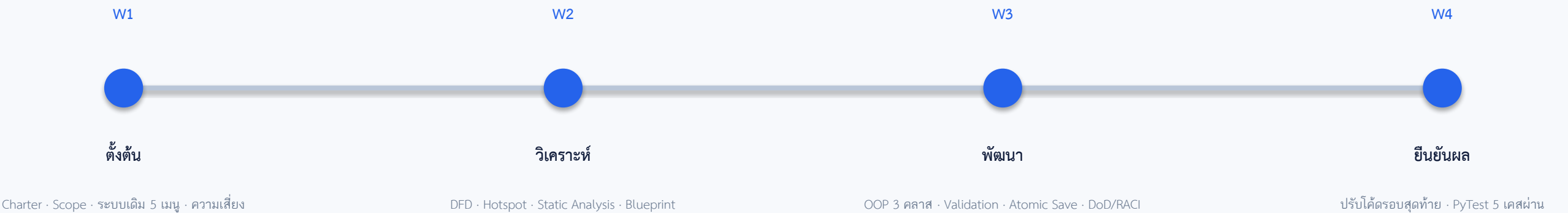
Tech Lead / Architect

เป้าหมายหลัก

- ▶ กำหนด Project Charter / Scope และความเสี่ยง
- ▶ วิเคราะห์ DFD, hotspot และ static quality ของระบบเดิม
- ▶ ออกแบบ blueprint สำหรับ OOP + Member/Discount
- ▶ ลงมือ refactor, validation, atomic save และ PyTest

เส้นทาง W1-W4

งานจากการตั้งต้นโครงการไปจนถึงโค้ด refactor ที่ทดสอบซ้ำได้



ก่อนและหลังการ Refactor

As-Is: v1

- Monolithic + global state
- JSON เขียนทับตรง เสี่ยงข้อมูลเสีย
- ▶ ไม่มี validation ค่าติดลบ/ชนิดข้อมูล
- main() ซ้ำซ้อน: CC = 14



Sprint 1 Deliverable

- Product / InventoryManager / InventoryCLI
- Atomic write: temp + os.replace
- Validator แยกตามช่อง + กัน ValueError
- Business Logic แยกจาก CLI

Baseline คุณภาพโค้ด

ค่าก่อนปรับใช้เป็นฐานเปรียบเทียบ — ไม่อ้างอิงเป็นค่าหลังปรับ

99

Lines of Code

7.10

Pylint / 10

14

Cyclomatic Complexity

17

Branches

อ่านคำร่วมกัน

- main() มี statements 55 (>50) และ branch 17 (>12)
- ▶ หลัง refactor โครงสร้างดีขึ้นและทดสอบผ่าน แต่ Sprint 1 ไม่ได้วัด Pylint/CC ซ้ำ
- ▶ เกณฑ์เป้าหมายในแผนถัดไป: ลดความซับซ้อนและแยกหน้าที่ให้ชัด

ผลทดสอบและ EVM

ยืนยันทั้งด้านเทคนิคและภาพการบริหารรอบแรก

5/5

PyTest PASS

4,000

PV (THB)

3,000

EV (THB)

4,200

AC (THB)

อ่านค่าร่วมกัน

- $SV = -1,000$ THB → งานช้ากว่าแผน
- $CV = -1,200$ THB → ใช้ต้นทุนมากกว่าคุณค่าที่ได้
- ▶ สาเหตุหลัก: technical debt ซ่อนใน app_v1.py และเวลาแก้ merge conflict

สิ่งที่เรียนรู้จาก Sprint 1

ฐานที่ใช้กำหนดวิธีทำงานใน Sprint ถัดไป

คุณภาพ

DoD 5 ข้อ + RACI ทำให้การส่งมอบมีเกณฑ์ตรวจที่ชัดเจน

ความเสี่ยงข้อมูล

Atomic write ลดความเสี่ยง JSON พัง แต่ยังไม่มี transaction ระดับฐานข้อมูล

Backward Compatibility

รองรับคีย์เก่า n/q/p/c และค่า default เมื่อฟิลด์ขาด

งานยังไม่จบ

SQLite, Member tiers และ Checkout ยังเป็นแบบออกแบบ ต้องกลับมาทำจริง

Sprint Review & Handoff

สิ่งที่ยืนยันได้จากหลักฐาน และสิ่งที่ส่งต่อไปยังรอบถัดไป

ส่งมอบแล้ว

- OOP 3 คลาสและ separation of concerns
- Atomic save + input validation
- PyTest 5 เคสพร้อม test.json

ความเสี่ยง/ข้อจำกัด

- JSON ยังไม่มี transaction
- E501 ต่างใน snapshot ประวัติ
- ▶ ไม่มี formal sponsor sign-off

งานส่งต่อ

- ▶ พัฒนา SQLite + Member + Checkout
- ▶ รับ CR-01 / CR-02 และแก้ BUG-101
- ▶ เตรียม UAT และ release gate